ADVANCED PYTHON PROGRAMMING

UNIT V

Database Integration and ORM

Student Reference Guide

• Relational database concepts and SQL CRUD

• SQLite with Python's sqlite3 module

• Parameterized queries and SQL injection prevention

• Transactions, ACID, and indexes

• NoSQL concepts and MongoDB basics

• SQLAlchemy ORM, sessions, and connection pooling

• MySQL, PostgreSQL, and Django ORM overview

Definitions • Commands • Runnable examples • Common mistakes • Revision checklist

Contents

1. Database fundamentals

2. SQL CRUD

3. SQLite with Python

4. SQL injection and safe queries

5. Transactions and ACID

6. Indexes and query performance

7. ORM Frameworks: SQLAlchemy and Django

8. MongoDB and NoSQL

9. MySQL and PostgreSQL

10. Connection pooling

11. Integrated mini-project

12. Quick reference and self-assessment

1. Database fundamentals

A **database** is an organized collection of data. A **DBMS (Database Management System)** is software used to create, query, secure, and manage databases.

In a relational database, data is organized into **tables**. A table has **columns** (attributes) and **rows** (records). A **schema** describes structures, types, and constraints.

Term	Definition
Primary key	Column or set of columns that uniquely identifies each row; values must be unique and not null.
Foreign key	Constraint linking a column to a key in another (or the same) table.
Constraint	Rule enforced by the database, such as NOT NULL, UNIQUE, CHECK, PRIMARY KEY, or FOREIGN KEY.
Query	Instruction sent to a database, often written in SQL.
CRUD	Create, Read, Update, Delete—the four common data operations.

Database choices

Database	Type / common use
SQLite	Embedded relational database; easy local files and small apps; Python standard library includes sqlite3.
MySQL	Client/server relational DBMS, commonly used in web applications.
PostgreSQL	Client/server relational DBMS with rich SQL features and strong extensibility.
MongoDB	Document-oriented NoSQL database storing BSON documents in collections.

2. SQL CRUD

SQL (Structured Query Language) is used to define relational structures and query or modify relational data.

```
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    marks INTEGER NOT NULL CHECK (marks BETWEEN 0 AND 100)
);
```

```
INSERT INTO students (name, marks)
VALUES ('Anu', 85);

SELECT student_id, name, marks
FROM students
WHERE marks >= 80
ORDER BY marks DESC;

UPDATE students
SET marks = 95
WHERE student_id = 1;

DELETE FROM students
WHERE student_id = 1;
```

SQL command groups

Group	Common commands	Purpose
DDL	CREATE, ALTER, DROP	Define or change database structures.
DML	INSERT, UPDATE, DELETE	Modify data.
DQL	SELECT	Retrieve data (commonly grouped separately for learning).
TCL	COMMIT, ROLLBACK	Control transaction completion.
DCL	GRANT, REVOKE	Manage privileges in systems that support them.

Important: an UPDATE or DELETE without a WHERE clause can affect every row in the table. Check the predicate before running data-changing statements.

3. SQLite with Python

SQLite is a lightweight relational database engine that stores the database in a local file. Python's built-in `sqlite3` module provides database access without a separate package installation.

Complete CRUD example

```
import sqlite3

DB_PATH = "college.db"

def main():
    # Connection creates the file if it does not exist.
    with sqlite3.connect(DB_PATH) as connection:
        cursor = connection.cursor()

        cursor.execute("""
            CREATE TABLE IF NOT EXISTS students (
                student_id INTEGER PRIMARY KEY,
                name TEXT NOT NULL,
                marks INTEGER NOT NULL
            )
        """)

        # CREATE (insert). Placeholders safely bind values.
        cursor.execute(
            "INSERT INTO students (name, marks) VALUES (?, ?)",
            ("Anu", 85)
        )
        new_id = cursor.lastrowid

        # READ
        cursor.execute(
            "SELECT student_id, name, marks FROM students WHERE student_id = ?",
            (new_id,)
        )
        print("Inserted:", cursor.fetchone())

        # UPDATE
        cursor.execute(
            "UPDATE students SET marks = ? WHERE student_id = ?",
            (95, new_id)
        )
        print("Updated rows:", cursor.rowcount)

        # READ all
        cursor.execute("SELECT * FROM students ORDER BY student_id")
        for row in cursor.fetchall():
            print(row)

        # DELETE
        cursor.execute(
            "DELETE FROM students WHERE student_id = ?",
            (new_id,)
        )
        print("Deleted rows:", cursor.rowcount)

    # Exiting the connection context commits if successful,
    # or rolls back on an exception. The connection object
    # itself should still be closed explicitly or with closing().
    connection.close()

if __name__ == "__main__":
    main()
```

A simpler, explicit connection lifecycle

```
import sqlite3
from contextlib import closing

with closing(sqlite3.connect("college.db")) as connection:
    with connection:  # transaction: commit or rollback
        connection.execute(
            "INSERT INTO students (name, marks) VALUES (?, ?)",
            ("Ravi", 78)
        )
# closing() guarantees connection.close(). Needs the students table created above.
```

Note on connection context managers

For sqlite3, `with connection:` manages a transaction but does not itself close the connection. Use explicit `close()` or `contextlib.closing()` when you want guaranteed closure.

Useful sqlite3 methods

Method / attribute	Meaning
connect(path)	Open or create a SQLite database connection.
connection.execute(sql, params)	Execute a SQL statement with optional bound parameters.
fetchone()	Fetch one result row or None.
fetchall()	Fetch all remaining result rows into a list.
lastrowid	Row ID of the last successful insert where supported.
rowcount	Number of rows affected/reported by a statement; behavior can vary by statement/driver.
commit()	Commit the current transaction.
rollback()	Roll back the current uncommitted transaction.

4. SQL injection and safe queries

SQL injection occurs when untrusted input is treated as SQL code because it is concatenated into a query string. Use parameterized queries so values are passed separately from SQL syntax.

```
# UNSAFE: do not build queries like this.
name = input("Name: ")
query = f"SELECT * FROM students WHERE name = '{name}'"

# SAFE for sqlite3:
cursor.execute(
    "SELECT * FROM students WHERE name = ?",
    (name,)
)
```

For a single value, the tuple must include a trailing comma: `(name,)`. Placeholders are for values, not table names or column names. If a query must choose a column dynamically, map user choices to a fixed allowlist of trusted identifiers.

5. Transactions and ACID

A **transaction** groups database operations into a logical unit. A successful transaction is committed; an unsuccessful one can be rolled back before it is committed.

ACID property	Meaning
Atomicity	All operations in the transaction succeed together or the transaction is rolled back.
Consistency	A committed transaction preserves defined database rules and constraints.
Isolation	Concurrent transactions are separated according to the database's isolation behavior.
Durability	After commit succeeds, changes are designed to survive failures according to database guarantees and configuration.

```
import sqlite3

with sqlite3.connect("bank.db") as connection:
    try:
        connection.execute(
            "UPDATE accounts SET balance = balance - ? WHERE account_id = ?",
            (100, 1)
        )
        connection.execute(
            "UPDATE accounts SET balance = balance + ? WHERE account_id = ?",
```

```
            (100, 2)
        )
    except sqlite3.Error:
        # Optional: log here. Any exception leaving the with-block rolls back.
        raise
# On success the context manager commits; it does not close the connection.
```

Rollback rule: rollback can undo changes that are still uncommitted in the current transaction. A normal rollback after a successful commit cannot undo that committed transaction.

6. Indexes and query performance

An **index** is an auxiliary data structure that helps the database locate rows without scanning every row. Many relational indexes use B-tree-like structures, but implementation details depend on the database and index type.

```
CREATE INDEX idx_students_marks ON students(marks);

EXPLAIN QUERY PLAN
SELECT * FROM students WHERE marks >= 80;
```

Index trade-offs

• Indexes can speed up suitable filters, joins, and sorting operations.

• Indexes use storage and add work to INSERT, UPDATE, and DELETE operations.

• An index is not automatically useful for every query; selectivity, table size, statistics, and query shape matter.

• Avoid creating duplicate or unnecessary indexes. Inspect actual query plans and measure performance.

7. ORM Frameworks: SQLAlchemy and Django

ORM (Object-Relational Mapping) maps Python classes and objects to relational tables and rows. SQLAlchemy generates SQL for common operations, but SQL knowledge remains important.

Install

```
python -m pip install sqlalchemy
python -c "import sqlalchemy; print(sqlalchemy.__version__)"
```

Runnable SQLAlchemy 2.x CRUD example

```
from sqlalchemy import create_engine, String, Integer, select
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session

class Base(DeclarativeBase):
    pass

class Student(Base):
    __tablename__ = "students"

    student_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(100), nullable=False)
    marks: Mapped[int] = mapped_column(Integer, nullable=False)

engine = create_engine("sqlite:///orm_students.db", echo=False)
Base.metadata.create_all(engine)  # Creates missing tables; not migrations.

with Session(engine) as session:
    # CREATE
    student = Student(name="Anu", marks=85)
    session.add(student)
    session.commit()
    print("New ID:", student.student_id)

    # READ by primary key
    found = session.get(Student, student.student_id)
    print(found.name, found.marks)

    # UPDATE
    found.marks = 95
    session.commit()

    # READ using a select statement
    statement = select(Student).where(Student.marks >= 90)
    for item in session.scalars(statement):
        print(item.name, item.marks)

    # DELETE
    session.delete(found)
    session.commit()
```

Core SQLAlchemy concepts

Concept	Definition
Engine	Central database interface; manages dialect and typically connection pooling for client/server databases.
Declarative Base	Base class used to declare mapped models.
Mapped class	Python class mapped to a database table.
Session	Unit of work that tracks objects and coordinates queries and transactions.
add(obj)	Associate an object with the session; a new object is pending insertion.
flush()	Send pending SQL to the database within the current transaction; does not commit.
commit()	Flush pending work and commit the current transaction.
rollback()	Roll back the current uncommitted transaction and reset relevant session state.
delete(obj)	Mark a persistent object for deletion when changes are flushed.

Transaction-safe session pattern

```
from sqlalchemy.orm import Session

try:
    with Session(engine) as session:
        with session.begin():
            session.add(Student(name="Ravi", marks=78))
            session.add(Student(name="Meena", marks=91))
    # session.begin() commits on success and rolls back on exception.
except Exception:
    # Handle or log the failure appropriately in real applications.
    raise
```

Important: `Base.metadata.create_all()` creates missing tables but does not safely evolve an existing schema. Use a migration tool such as Alembic for controlled schema changes in a maintained application.

Django ORM overview

Django is a Python web framework that includes its own ORM. A model is a Python class that subclasses `models.Model`: the class maps to a table and each field attribute maps to a column. Queries are built with **QuerySets** through the model's manager (`objects`), and Django generates the SQL for the configured database: SQLite (the default), PostgreSQL, MySQL/MariaDB, or Oracle.

Install, create a project and an app

```
python -m pip install django
django-admin startproject collegesite
cd collegesite
python manage.py startapp academics
```

Then add `"academics"` to `INSTALLED_APPS` in `collegesite/settings.py`.

Define a model

```
# academics/models.py
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    marks = models.IntegerField()

    def __str__(self):
        return self.name
```

Django adds an auto-incrementing `id` primary key automatically unless you declare your own. Field types and options such as `max_length` determine the column type and constraints.

Create the table with migrations

```
python manage.py makemigrations academics
python manage.py migrate
```

`makemigrations` records model changes as migration files; `migrate` applies them to the database. Unlike SQLAlchemy's `create_all()`, migrations are built in; repeat both commands whenever a model changes.

CRUD with the QuerySet API

Run `python manage.py shell`, then:

```
from academics.models import Student

# CREATE
s = Student.objects.create(name="Anu", marks=85)

# READ
Student.objects.get(pk=s.pk)                      # exactly one row
Student.objects.filter(marks__gte=80).order_by("-marks")
Student.objects.all()

# UPDATE
s.marks = 95
s.save()
Student.objects.filter(name="Anu").update(marks=97)   # bulk UPDATE

# DELETE
s.delete()
```

QuerySet concept	Meaning
get(...)	Returns exactly one object; raises `Student.DoesNotExist` if none match and `MultipleObjectsReturned` if several do.
filter(...) / all()	Return a lazy QuerySet; SQL runs when you iterate it or call `list()` or `len()`.
marks__gte=80	Field lookup with a double underscore, meaning `marks >= 80`. Others include `__lt`, `__contains` and `__in`.
update(...)	Runs one bulk UPDATE statement; it does not call `save()` or send model signals.

Transactions and indexes

By default Django runs in autocommit mode, so each query is committed immediately unless it is inside an `atomic()` block.

```
from django.db import transaction

with transaction.atomic():
    Student.objects.create(name="Ravi", marks=78)
    Student.objects.create(name="Meena", marks=91)
# Commits if the block succeeds; rolls back if an exception escapes.

# Index on a frequently filtered column (inside the model class):
class Meta:
    indexes = [models.Index(fields=["marks"])]
```

Run `makemigrations` and `migrate` again after adding an index. `db_index=True` on a field is the shorthand for a single-column index.

Database settings

```
# settings.py (add "import os" at the top for the PostgreSQL example)

# SQLite - the default created by startproject
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}

# PostgreSQL - needs: python -m pip install "psycopg[binary]"
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "college",
        "USER": os.environ["DB_USER"],
        "PASSWORD": os.environ["DB_PASSWORD"],
        "HOST": "localhost",
        "PORT": "5432",
    }
}
```

For MySQL, use `django.db.backends.mysql` with the `mysqlclient` driver. Keep credentials in environment variables, never in source control.

Connection management and pooling

By default Django closes the database connection at the end of each request (`CONN_MAX_AGE` is `0`). Setting `CONN_MAX_AGE` to a number of seconds keeps a connection open for reuse (persistent connections, all backends). Since Django 5.1, PostgreSQL also supports a real connection pool through `psycopg_pool`; Oracle gained pooling in Django 5.2.

```
# Option A - persistent connections (any backend): reuse a connection for 10 minutes
DATABASES["default"]["CONN_MAX_AGE"] = 600

# Option B - built-in connection pool (PostgreSQL, Django 5.1+)
# Needs: python -m pip install "psycopg[binary,pool]"
# Use it instead of option A; CONN_MAX_AGE must stay 0 when the pool is enabled.
DATABASES["default"]["OPTIONS"] = {"pool": True}
```

SQLAlchemy and Django ORM compared

Topic	SQLAlchemy	Django ORM
Scope	Stand-alone toolkit and ORM for any Python program	ORM that is part of the Django framework
Model base	`DeclarativeBase` subclass with `mapped_column`	`models.Model` subclass with field classes
Saving changes	`Session` tracks objects; you call `commit()`	`save()` / `delete()` run immediately (autocommit)
Queries	`select(Student).where(...)` with `session.scalars()`	`Student.objects.filter(...)` (QuerySet)
Schema changes	`create_all()` for new tables; Alembic for migrations	`makemigrations` and `migrate` (built in)
Transactions	`session.begin()`, `commit()`, `rollback()`	`transaction.atomic()`

8. NoSQL and MongoDB basics

NoSQL is a broad category of database designs that are not limited to the traditional relational-table model. It includes document, key-value, wide-column, and graph databases. NoSQL does not simply mean 'no SQL' or 'no structure'.

Relational model	MongoDB model
Database	Database
Table	Collection
Row / record	Document
Column	Field
Primary key	The _id field; MongoDB creates one if omitted.

MongoDB stores documents in BSON, a binary representation related to JSON that supports additional types. Documents in a collection can have different fields, although applications often enforce their own schema rules.

Install Python driver

```
python -m pip install pymongo
```

This installs the driver, not the MongoDB server. A MongoDB server must be installed, running, and reachable at the connection URI.

PyMongo CRUD example (requires local MongoDB on port 27017)

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/", serverSelectionTimeoutMS=3000)

try:
    # Verify that the server can be reached.
    client.admin.command("ping")

    db = client["college"]
    students = db["students"]

    # CREATE
    result = students.insert_one({"name": "Anu", "marks": 85})
    student_id = result.inserted_id

    # READ
    print(students.find_one({"_id": student_id}))

    # UPDATE only the marks field
    update_result = students.update_one(
        {"_id": student_id},
        {"$set": {"marks": 95}}
    )
    print(update_result.matched_count, update_result.modified_count)

    # DELETE
    delete_result = students.delete_one({"_id": student_id})
    print("Deleted:", delete_result.deleted_count)
finally:
    client.close()
```

MongoDB operators and result counts

Operation	Meaning
insert_one(document)	Insert one document.
find_one(filter)	Return one matching document or None.
find(filter)	Return a cursor over matching documents.
update_one(filter, {'$set': ...})	Update selected fields in one matching document.
replace_one(filter, replacement)	Replace the contents of one matching document, keeping its _id behavior subject to MongoDB rules.
delete_one(filter)	Delete one matching document.
matched_count	Number of documents matched by an update.
modified_count	Number of documents actually modified.

Use a unique filter such as _id when exactly one known document should be changed. A filter such as {'name': 'Anu'} can match multiple people, and update_one() changes only one matching document.

9. MySQL and PostgreSQL

MySQL and PostgreSQL are client/server relational database systems. Unlike SQLite, they normally run as separate server processes and accept connections over a network or local socket.

Typical setup outline

- Install and start the database server, then create a database and a least-privilege application user.
- Install a Python driver appropriate to the database and version.
- Keep credentials outside source code, for example in environment variables or a secrets manager.
- Use parameterized statements or an ORM; do not concatenate untrusted values into SQL.

• Configure network access, backups, permissions, migrations, and connection limits for real deployments.

Example installation commands (driver packages)

```
# MySQL driver option
python -m pip install mysql-connector-python

# PostgreSQL driver option
python -m pip install "psycopg[binary]"

# SQLAlchemy can work with supported drivers:
python -m pip install sqlalchemy
```

Connection URI examples (use environment variables for real credentials)

```
# SQLAlchemy URL examples:
# MySQL:      mysql+mysqlconnector://user:password@localhost:3306/college
# PostgreSQL: postgresql+psycopg://user:password@localhost:5432/college
# SQLite:     sqlite:///college.db
```

> Driver and URL details can vary by database and driver version. Never commit real credentials to Git.

10. Connection pooling

Connection pooling reuses database connections rather than repeatedly creating new ones. Establishing a client/server connection can involve networking, authentication, and session setup, so reuse can reduce overhead.

How the lifecycle works

> Application request → acquire a connection from the pool → execute queries → return the connection to the pool when released. The connection may later be reused by another request.

• SQLAlchemy's Engine normally owns the connection pool for client/server databases.

• Create an Engine once for the application lifetime rather than once per query.

• Close or exit a Session scope so checked-out connections are returned to the pool.

• Pooling is different from indexing: pooling improves connection reuse; indexes can improve data lookup.

• Pools have limits and configuration; they do not guarantee that a database cannot be overloaded. SQLite pooling behavior differs from many client/server databases.

11. Integrated mini-project: SQLite student repository

This example packages common database operations into reusable functions. Save it as `student_repository.py` and run it with Python. The database file is created locally.

```
# student_repository.py
import sqlite3
from contextlib import closing

DB_PATH = "students_app.db"

def initialize_database():
    with closing(sqlite3.connect(DB_PATH)) as connection:
        with connection:
            connection.execute("""
                CREATE TABLE IF NOT EXISTS students (
                    student_id INTEGER PRIMARY KEY,
                    name TEXT NOT NULL,
                    marks INTEGER NOT NULL CHECK (marks BETWEEN 0 AND 100)
                )
            """)

def add_student(name, marks):
    with closing(sqlite3.connect(DB_PATH)) as connection:
        with connection:
            cursor = connection.execute(
                "INSERT INTO students (name, marks) VALUES (?, ?)",
                (name, marks)
```

```
            )
            return cursor.lastrowid

def get_student(student_id):
    with closing(sqlite3.connect(DB_PATH)) as connection:
        row = connection.execute(
            "SELECT student_id, name, marks FROM students WHERE student_id = ?",
            (student_id,)
        ).fetchone()
        return row

def update_marks(student_id, marks):
    with closing(sqlite3.connect(DB_PATH)) as connection:
        with connection:
            cursor = connection.execute(
                "UPDATE students SET marks = ? WHERE student_id = ?",
                (marks, student_id)
            )
            return cursor.rowcount == 1

def delete_student(student_id):
    with closing(sqlite3.connect(DB_PATH)) as connection:
        with connection:
            cursor = connection.execute(
                "DELETE FROM students WHERE student_id = ?",
                (student_id,)
            )
            return cursor.rowcount == 1

if __name__ == "__main__":
    initialize_database()
    student_id = add_student("Anu", 85)
    print("Created:", get_student(student_id))
    print("Updated:", update_marks(student_id, 95))
    print("After update:", get_student(student_id))
    print("Deleted:", delete_student(student_id))
```

Run it

```
python student_repository.py
```

12. Unit V quick reference and self-assessment

Need	Use
Local relational database	sqlite3.connect('file.db')
Insert with parameters	execute('INSERT ... VALUES (?, ?)', (value1, value2))
Fetch one row	fetchone()
Fetch all rows	fetchall()
Commit / undo uncommitted work	commit() / rollback()
Prevent SQL injection	Parameterized queries; allowlist dynamic identifiers.
Inspect SQLite query plan	EXPLAIN QUERY PLAN SELECT ...
ORM for relational data	SQLAlchemy mapped classes and Session.
MongoDB Python driver	pymongo.MongoClient(...)
Connection reuse	One long-lived SQLAlchemy Engine with a configured pool where supported.

- I can explain tables, rows, columns, primary keys, foreign keys, and constraints.
- I can write SQL INSERT, SELECT, UPDATE, and DELETE statements safely.
- I can implement SQLite CRUD using sqlite3 and parameterized queries.
- I can explain SQL injection, transactions, ACID, commit, and rollback.

- I can explain index benefits and write costs, and inspect a query plan.
- I can describe Engine, Session, mapped classes, flush, commit, and rollback in SQLAlchemy.
- I can distinguish a MongoDB collection from a document and perform CRUD using PyMongo.
- I can describe the purpose of MySQL/PostgreSQL drivers and connection pooling.
- I can explain why schema migrations, least-privilege users, backups, and secret management matter.
- I can define a Django model, run migrations, and perform CRUD with the QuerySet API.

Practice tasks

- Create a courses table with a primary key and a NOT NULL course name.
- Write a function that finds a student by ID and returns None if no row exists.
- Demonstrate how parameterized SQL prevents input from becoming query syntax.
- Use a transaction to update two related rows; verify rollback on a forced error.
- Create an index on a frequently filtered column and inspect EXPLAIN QUERY PLAN.
- Extend the SQLAlchemy model with a department field and add a query by department.
- Use MongoDB to insert two documents and update only one selected by _id.
- Explain when SQLite is enough and when a client/server database may be more appropriate.
- Create a Django `Student` model with a department field, run the migrations, and list one department's students with `filter()`.